



JARAPP

Manual Técnico

Arquitectura, servicios y guía de desarrollo - 2026



Contenido del Manual

#	Sección	Pág.
1	Stack tecnológico	3
2	Variables de entorno	4
3	Arquitectura del proyecto	5
4	Schema de base de datos	6
5	Capa de servicios (src/services/)	8
6	RBAC — Control de acceso por roles (auth.js)	9
7	PWA — Service Worker e iOS	10
8	Funciones SQL personalizadas	11
9	Deploy CI/CD — Netlify	12
10	Consideraciones técnicas críticas	13



1. Stack tecnológico

Capa	Tecnología	Versión	Rol en el proyecto
Frontend	Vanilla JavaScript (ES Modules)	—	Lógica de la aplicación sin frameworks
Build	Vite	8.0.3	Bundler ultrarrápido con HMR y tree-shaking
Base de datos	Supabase	2.101.1	PostgreSQL + Auth + Storage + RLS + Realtime
Gráficas	Chart.js	4.5.1	Visualizaciones del Dashboard 360°
Íconos	Lucide	1.7.0	Sistema de íconos SVG consistente
PDF	pdfmake	0.3.9	Generación de PDFs para el Cotizador
Excel	xlsx (SheetJS)	0.18.5	Exportación de reportes a Excel
HTTP	axios	1.14.0	Peticiones HTTP auxiliares
IA / LLM	Groq API	—	JaraBot — llama-3.3-70b-versatile
TRM	open.er-api.com	—	Tipo de cambio USD/COP actualizado diario
Hosting	Netlify	—	Deploy automático desde GitHub (main)
CSS	Custom Properties + tokens.css	—	Design tokens, temas claro/oscuro
PWA	Service Worker + Web Manifest	—	Offline support + instalable en iOS/Android



2. Variables de entorno

Variable	Descripción	Ejemplo / Formato
VITE_SUPABASE_URL	URL del proyecto Supabase	https://xxx.supabase.co
VITE_SUPABASE_KEY	Anon Key (pública) de Supabase	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
VITE_GROQ_API_KEY	API Key de Groq para JaraBot	gsk_...

Las variables se configuran en Netlify → Site Settings → Environment Variables para producción. Para desarrollo local, crear un archivo .env en la raíz del proyecto. NUNCA commitear el archivo .env al repositorio de GitHub. Las credenciales de Supabase también tienen un fallback hardcoded en src/db.js para garantizar funcionamiento en cualquier dispositivo sin configuración.

Cargar en desarrollo local:

Crear archivo .env en la raíz del proyecto con las tres variables. Ejecutar npm run dev — Vite las inyecta automáticamente como import.meta.env.VITE_*



3. Arquitectura del proyecto

Ruta	Descripción
index.html	Entry point HTML con meta tags PWA, iOS y theme-color
src/main.js	Bootstrap, router SPA, NAV_GROUPS, badges, init PWA
src/db.js	Cliente Supabase — credenciales en cascada (env → hardcoded)
src/auth.js	RBAC completo: ROLE_TEMPLATES, clase Auth, persistencia sesión
src/utlis.js	Helpers globales: formatCOP, showToast, downloadExcel, paginación
src/style.css	Estilos globales + optimizaciones PWA iOS + safe areas
src/tokens.css	Design tokens: colores, tipografía, radios, sombras
src/views/dashboard.js	Dashboard 360° con viaje, alertas, KPIs y reportes
src/views/sales.js	Ventas: CRUD, abonos, modal detalle, exportación Excel
src/views/inventory.js	Inventario: catálogo, stock, precios, fotos Supabase Storage
src/views/logistics.js	Seguimientos: fases logísticas 1-7, historial, alertas
src/views/purchases.js	Compras USA: órdenes, estados, vinculación con seguimientos
src/views/viaje.js	Viaje EEUU: gastos, distribución, balance personal/negocio
src/views/cotizador.js	Cotizador con TRM en vivo, PDF cliente y PDF interno
src/views/clients.js	Clientes: CRUD, historial de compras, cartera pendiente
src/views/finance.js	Finanzas: egresos por categoría, balance de caja
src/views/calculadora.js	Calculadora de precios con desglose completo
src/views/params.js	Parámetros configurables: metas, días alerta, TRM manual
src/views/admin.js	Admin: gestión de usuarios y roles (solo Admin)
src/views/settings.js	Configuración: URL Supabase, logo empresa, ajustes
src/services/trm.js	TRM Service: obtener tipo de cambio con caché y fallback
src/services/alertas.js	Alertas Service: 5 tipos de alertas con caché 30 min
src/services/viajes.js	Viaje Service: getActivo(), CRUD de viajes y gastos
src/services/config.js	Config Service: parámetros del sistema desde Supabase
src/services/jarabot.js	JaraBot Service: construcción de contexto y llamada Groq API
src/components/jarabot-widget.js	Widget de chat flotante con historial de sesión
src/components/alertas-panel.js	Panel renderizador de alertas con links de navegación
src/dashboard/	4 módulos lazy del Dashboard 360° (ventas, logística, finanzas, clientes)
public/sw.js	Service Worker v2: caché offline, network-first/cache-first
public/manifest.json	PWA manifest: íconos, colores, scope, orientación, lang



Ruta	Descripción
public/icon-192.png	Ícono PWA 192×192 generado con scripts/generate-icons.cjs
public/icon-512.png	Ícono PWA 512×512 generado con scripts/generate-icons.cjs
scripts/generate-icons.cjs	Generador de íconos PNG puro Node.js (sin dependencias externas)



4. Schema de base de datos (Supabase / PostgreSQL)

Tablas principales:

Tabla	Columnas principales	Notas
Ventas	id (uuid), fecha, cliente_id, productos (jsonb), total_cop, saldo_pendiente, estado, tipo (encargo/local), numero_factura, fecha_factura_usd, trm_cop, tracking	Estado: activa / entregada / cancelada
Clientes	id (uuid), nombre, telefono, ciudad, instagram, email, notas, activo	cliente_id en Ventas es FK a id
Productos	id (uuid), nombre, marca, categoria, precio_usd, precio_cop, stock_medellin, stock_usa, activo, imagen_url, descripcion, peso_lb	imagen_url → Supabase Storage bucket jarapo-images
Logistica	id (texto), fase (1-7), tracking_usa, tracking_col, fecha_compra, fecha_llegada_col, producto, compra_id, notas, peso_lb, costo_usd, dias_alerta	ID formato: {timestamp_ms}LOG
Compras	id (uuid), fecha, proveedor, producto, cantidad, precio_usd, total_usd, estado, tracking, notas, logistica_id, viaje_id	Vinculada con Logistica y viajes
Gastos	id (uuid), fecha, categoria, descripcion, monto_cop, monto_usd, trm, proveedor, comprobante_url	Gastos operativos del negocio en Colombia
Abonos	id (uuid), venta_id, fecha, monto_cop, metodo_pago, notas	FK venta_id → Ventas.id

Tablas del sistema:

Tabla (snake_case)	Descripción
viajes	Viajes a EEUU con campos de gastos y distribución
trm_historico	Historial de TRM diaria — fecha, valor, fuente
user_profiles	Perfiles de usuario con rol, nombre, email, activo
login_logs	Registro de inicios de sesión con IP, fecha, dispositivo
params	Parámetros configurables del sistema (key-value)

Convención de nombres: tablas de negocio en PascalCase (Ventas, Clientes, Productos, Logistica, Compras, Gastos, Abonos). Tablas del sistema en snake_case (viajes, trm_historico, user_profiles, login_logs).



5. Capa de servicios (src/services/)

```
Todos los servicios usan el patrón: import { db } from "../db.js"; const client = () => db.client;
```

TrmService (trm.js)

- `get()` — TRM de hoy con caché 24h, fuente `open.er-api.com`
- `getHistorico(días)` — últimos N días desde `trm_historico`
- `guardar(valor, fuente)` — persiste nueva TRM en Supabase

AlertasService (alertas.js)

- `getAlertas(forceRefresh)` — retorna todas las alertas activas (caché 30 min)
- `invalidar()` — limpia el caché para forzar recarga
- `hayDanger()` — true si hay alguna alerta nivel danger
- `dangerCount()` — cantidad de alertas danger activas

ViajeService (viajes.js)

- `getActivo()` — viaje con `fecha_fin` null y `activo=true`
- `getAll()` — todos los viajes ordenados por `fecha_inicio` DESC
- `crear(data)` — nuevo viaje
- `actualizar(id, data)` — actualizar campos de gastos o estado
- `cerrar(id)` — setea `fecha_fin = today` y `activo = false`

ConfigService (config.js)

- `get(key)` — obtener parámetro por clave desde `params` table
- `set(key, value)` — actualizar o insertar un parámetro
- `getAll()` — todos los parámetros del sistema
- `getLogo()` — URL del logo desde `sessionStorage` o Supabase Storage

JaraBotService (jarabot.js)

- `buildContext(role)` — consulta todos los datos del negocio en paralelo
- `chat(message, role)` — llama a Groq API con contexto real del negocio
- Modelo: `llama-3.3-70b-versatile`
- Endpoint: `https://api.groq.com/openai/v1/chat/completions`



6. RBAC — Control de acceso por roles (auth.js)

Métodos principales de la clase Auth:

Método	Descripción
signIn(email, pwd)	Login con Supabase Auth — devuelve sesión y perfil de usuario
signOut()	Cierra sesión y limpia localStorage y sessionStorage
getSession()	Retorna la sesión activa desde localStorage (sin llamada remota)
getProfile()	Lee el perfil del usuario (rol, nombre) desde user_profiles
can(module)	Verifica si el rol del usuario puede acceder al módulo
isAdmin()	true si el rol es "admin"
isGerente()	true si el rol es "admin" o "gerente"

ROLE_TEMPLATES — módulos permitidos por rol:

Módulo	Admin	Gerente	Ventas	Logística	Finanzas	Viewer
dashboard	✓	✓	✓	✓	✓	✓
inventory	✓	✓	✓ (ver)	✓	—	—
sales	✓	✓	✓	—	✓ (ver)	—
cotizador	✓	✓	✓	—	—	—
purchases	✓	✓	—	✓	—	—
logistics	✓	✓	—	✓	—	—
viaje	✓	✓	—	—	—	—
clients	✓	✓	✓	—	—	—
finance	✓	✓	—	—	✓	—
calculadora	✓	✓	—	✓	✓	—
params	✓	—	—	—	—	—
admin	✓	—	—	—	—	—
settings	✓	—	—	—	—	—



7. PWA — Service Worker e iOS

Estrategias de caché del Service Worker (public/sw.js):

Tipo de recurso	Estrategia	Cachés utilizado
JS, CSS, imágenes, fuentes (origin)	Cache First → Network fallback	jarapp-static-v2
Supabase API (*.supabase.co)	Network First → Cache fallback	jarapp-dynamic-v2
HTML / navegación	Network First → Cache fallback	jarapp-dynamic-v2
APIs externas (Groq, TRM)	Network Only (no se cachea)	—
Offline fallback de navegación	Retorna index.html cacheado	jarapp-static-v2

Meta tags iOS en index.html:

Meta tag	Valor	Propósito
apple-mobile-web-app-capable	yes	Modo standalone en iOS (elimina barra Safari)
apple-mobile-web-app-status-bar-style	default	Barra de estado del sistema visible
apple-mobile-web-app-title	JARAPP	Nombre de la app en pantalla de inicio iOS
apple-touch-startup-image	/icon-512.png	Splash screen al abrir la app
apple-touch-icon (180×180)	/icon-192.png	Ícono en pantalla de inicio iOS
viewport viewport-fit=cover	cover	Contenido bajo notch / Dynamic Island
theme-color light	#E63946	Color de barra del navegador en modo claro
theme-color dark	#1D3557	Color de barra del navegador en modo oscuro

El SW maneja mensajes CLEAR_CACHE (limpia todos los cachés) y SKIP_WAITING (activa el nuevo SW inmediatamente). El indicador offline usa window.addEventListener ("offline"/"online") y muestra un banner en la parte inferior con safe-area-inset-bottom.



8. Funciones SQL personalizadas

Funciones PL/pgSQL en Supabase:

Función	Parámetros	Retorna	Uso
get_ventas_resumen()	p_desde date, p_hasta date	TABLE(total, cobrado, cartera, cobrada)	KPIs financieros del Dashboard
get_logistica_fases()	p_activas boolean	TABLE(fase, count, dias_procesado)	Estado del pipeline logístico

Índices para optimización de consultas:

Índice	Tabla	Columnas	Beneficio
idx_ventas_estado	Ventas	estado, fecha	Filtrar ventas activas por fecha
idx_logistica_fase	Logistica	fase, id	Ordenar seguimientos por fase y tiempo
idx_abonos_venta	Abonos	venta_id, fecha	Calcular saldo pendiente por venta
idx_trm_fecha	trm_historico	fecha DESC	Obtener TRM más reciente eficientemente
idx_user_profiles_role	user_profiles	role, activo	Filtrar usuarios por rol activo

Patrón de consulta flat obligatorio: las consultas de Supabase NO usan joins anidados (las FKs no están declaradas en el schema). Siempre se hacen 3 consultas planas y se combinan manualmente con Map en JavaScript.



9. Deploy CI/CD — Netlify

Configuración de Netlify (.netlify/netlify.toml):

- Build command: npm run build
- Publish directory: dist/
- Node version: 18 (configurado en variables de entorno de Netlify)
- SPA redirect: /* → /index.html con código 200 (reescritura, no redirección)
- Deploy automático: cada push a rama main dispara un build en Netlify
- Preview deploys: cada PR genera un URL de preview único

Comandos de desarrollo:

Comando	Descripción
npm run dev	Servidor de desarrollo con HMR en http://localhost:5173
npm run build	Build de producción — genera carpeta dist/
npm run preview	Sirve el build de producción localmente para validar
node scripts/generate-icons.cjs	Genera icon-192.png y icon-512.png en public/
python scripts/generate_manuals.py	Genera los dos manuales PDF en docs/

Netlify usa caché de node_modules entre builds para reducir tiempos. Las variables de entorno VITE_* se inyectan durante el proceso de build de Vite — no en runtime del navegador. Cambiar una variable de entorno requiere un nuevo deploy.



10. Consideraciones técnicas críticas

Patrón de servicio DB obligatorio

Siempre usar: `import { db } from "../db.js"; const client = () => db.client;` NUNCA importar supabase directamente. El módulo `db.js` implementa el fallback de credenciales (`env` → `hardcoded`) de forma centralizada.

Consultas flat — no usar joins anidados

Las foreign keys no están declaradas en el schema de Supabase. Las queries con relaciones anidadas devuelven error. Siempre hacer 3 queries planas y combinar con `Map` en JS.

Formato de IDs de Logística

El ID de Logística tiene el formato `{timestamp_ms}LOG` — el prefijo numérico es el Unix timestamp en milisegundos. Esto permite ordenar cronológicamente con un simple `ORDER BY id DESC` sin necesidad de un campo fecha.

CSS — variable correcta para texto tenue

Usar `--text-faint` para texto tenue (gris suave). La variable `--text-tertiary` NO existe. Verificar siempre en `tokens.css`.

package.json tiene "type": "module"

Todos los archivos `.js` se tratan como ES Modules. Los scripts de Node.js que usen CommonJS (`require/module.exports`) DEBEN tener extensión `.cjs` para funcionar correctamente.

State pattern de módulos

Cada módulo de vista usa: `let _rl = null; let _nav = null; + función _render()`. Los event handlers se exponen como `window._nombreFuncion()` para ser accesibles desde atributos `onclick` en el HTML generado dinámicamente.

Persistencia de badge Viaje Activo en sidebar

El badge "ACTIVO" del sidebar se bake en el HTML durante `renderLayout()` leyendo `sessionStorage.getItem("JARAPP_VIAJE_ACTIVO")` de forma síncrona. Esto evita que desaparezca en cada re-render del nav. `window._actualizarBadgeViaje()` maneja las actualizaciones asíncronas.